

```
int a = 5;
a = 10 - a * 6 % 7 + 14 / 3; // a * 6 is done as its priority is high from left to right. (Exp - 1)
a = 10 - 30 % 7 + 14 / 3; // 30 % 7 is done next
a = 10 - 2 + 14 / 3; // 14 / 3 is next.
a = 10 - 2 + 4; // 10 - 2 is next.
a = 8 + 4; // 8 + 4 is next.
a = 12;
```

Fig. 2: C statement with arithmetic operators (multiplicative and additive) operators

```
int a = 5;
a = 12 * 10 - 4 & a * 6 % 7 + 16 / 3 | 1; // Now follow M-A-x-x-x-A-R-O (Exp - 2) from L to R
a = 12 * 10 - 4 & 30 % 7 + 16 / 3 | 1; // after a * 6; 30 % 7 is next
a = 12 * 10 - 4 & 2 + 16 / 3 | 1; // 16 / 3 is next
a = 12 * 10 - 4 & 2 + 5 | 1; // 10 - 4 is next
a = 12 * 6 & 2 + 5 | 1; // 2 + 5 is next
a = 12 * 6 & 7 | 1; // bitwise AND is next, irrespective of place.
a = 12 & 6 | 1; // bitwise EXOR is next.
a = 10 | 1; // bitwise OR is next.
a = 11;
```

Fig. 3: Statement with multiplicative, additive, AND, EXOR and OR operators

```
int a = 5;
a = 9 < < 2 ^ a - a & a * 2 % 3 + 2 | 9 / 2 > > 1 < < 2; // M-A-S-x-x-A-R-O
a = 9 < < 2 ^ a - a & 10 % 3 + 2 | 9 / 2 > > 1 < < 2; // First ax2 is done, next 10 % 3
a = 9 < < 2 ^ a - a & 1 + 2 | 9 / 2 > > 1 < < 2; // next 9 / 2
a = 9 < < 2 ^ a - a & 1 + 2 | 4 > > 1 < < 2; // next Subtraction.
a = 9 < < 2 ^ 0 & 1 + 2 | 4 > > 1 < < 2; // next addition
a = 9 < < 2 ^ 0 & 3 | 4 > > 1 < < 2; // Shift left operation
a = 36 * 0 & 3 | 4 > > 1 < < 2; // Shift Right operation
a = 36 * 0 & 3 | 2 < < 2; // Again shift Left
a = 36 * 0 & 3 | 8; // Bitwise AND
a = 36 * 0 | 8; // Bitwise EXOR
a = 36 | 8; // Bitwise OR
a = 44;
```

Fig. 4: With multiplicative, additive, AND, OR, EXOR and shift operators

```
int a = 6, b = 3; // We have M-A-S-G-E-A-R-O in the expression.
a = a / 2 = b | a < < a / b + b | a * b ^ b < a;
a = 6 / 2 = 3 | 6 < < 6 / 3 + 3 | 6 * 3 ^ 3 < 6; // First Division (6 / 2)
a = 3 = 3 | 6 < < 6 / 3 + 3 | 6 * 3 ^ 3 < 6; // Next Division (6 / 3)
a = 3 = 3 | 6 < < 2 + 3 | 6 * 3 ^ 3 < 6; // Next Multiplication (6x3)
a = 3 = 3 | 6 < < 2 + 3 | 18 ^ 3 < 6; // Next is Addition (2 + 3)
a = 3 = 3 | 6 < < 5 | 18 ^ 3 < 6; // Next is Shift left (6 < 5)
a = 3 = 3 | 192 | 18 ^ 3 < 6; // Next is Less than (3 < 6)
a = 3 = 3 | 192 | 18 ^ 3; // Next is Equality Check (3 = 3)
a = 1 | 192 | 18 ^ 1; // Next is Bitwise EXOR (18 ^ 1)
a = 1 | 192 | 19; // Next is Bitwise OR (1 | 192)
a = 193 | 19; // Next is Bitwise OR (193 | 19)
a = 211
```

Fig. 5: With multiplicative, additive, relational, bit-wise - &, |, ^ and shift operators

hand, if you are shifting N by k bits towards the right, you are dividing the number by 2^k .

That means, for an integer N,
 $N < < k \Rightarrow N \times 2^k$ // Power-efficient multiplication
 $N > > k \Rightarrow N / 2^k$ // Power-efficient division

Let us add relational operators of the two operands and try to get the value of the expression shown in Fig. 5 by following the priority order of the operator rhyme. Note that, within the available relational operators, there exist two priority levels among equality operators (low) and greater or less than operators (high).

Next, let us observe the behaviour of compound conditional AND (&&) and compound conditional OR (||) statements, too. Among these, logical AND (&&) has higher priority compared to logical OR (||). In the expression given in Fig. 6, again, by following the operator's priority rhyme, the step-by-step value is obtained.

Assignment operator (=) is the last priority in the rhyme. Note that, other than the regular assignment operator, a combination of assignment with arithmetic operators or with bit-wise operators too may fall under this observation. For example, consider the expression in Fig. 7. After evaluating the complete expression on the left-hand side, 63 is the result. Next, we have an assignment operator concatenated with an addition operator; this addition will be performed in the end, yielding the final value as 69.

Finally, consider the expression shown in Fig. 8, where most operators are deliberately used to understand how to evaluate to a final value, by following the order of priority as per the rhyme.

Although the expressions given in Figs 5, 6 and 7 are vague in nature—which a developer rarely adopts, for the other expressions, especially expressions that can be a combination of multiplicative,